lierda 利尔达科技集团 | 证券代码 832149

nRF52840+GH3X2X 软件移植说明

—— 基于 NCS-SDK

日期：2022/10/25

版本：V1.00

撰写：梁钊华

利尔达科技集团股份有限公司
LIERDA SCIENCE & TECHNOLOGY GROUP CO.,LTD

目 录

说明 1
1. GH3X2X 硬件资源和软件资源要求。 1
1.1 GH3X2X SPI 接口 2
1.2 GH3X2X I2C 接口 2
1.3 GH3X2X 中断接口 3
1.4 GH3X2X 复位接口 3
2. GH3X2X 驱动&算法库移植 3
2.1 添加驱动&算法库文件和例程代码到工程 3
2.2 添加底层驱动代码 4
2.2.1 Log 打印接口 4
2.2.2 SPI 接口 4
2.2.3 I2C 接口 7
2.2.4 延时接口 9
2.2.5 复位接口 9
2.2.6 中断接口 9
2.2.7 与上位机通信接口 10
2.2.8 软件佩戴检测定时器接口 12
2.2.9 G-sensor 数据获取接口（示例） 12
2.3 修改配置文件 13
2.4 添加寄存器配置表 14
2.5 应用层调用上层接口 14
2.6 测试验证 16
2.6.1 正常初始化的 LOG 信息输出 16
2.6.2 通信异常的初始化 LOG 信息 17
2.6.3 测试采样功能是否正常 17
附 录 17
1. CMake 配置修改参考。 17

说明

为便于客户开发 GH3X2X 相关产品，使用汇顶研发的算法库，更方便的集成调试相关功能，汇顶提供了 GH3X2X 配套的驱动库、算法库以及对应的源码。

GH3X2X 驱动库集成于 Host 端，Host 端通过 GH3X2X 驱动库，可以直接控制 GH3X2X 芯片启动或停止采样、配置更新、监测 GH3X2X 中断并按对应事件处理；同时可以通过驱动库无缝传输数据到算法库进行算法运算，目前支持的算法有 HR、SPO2、HRV、NADT 和 ECG（仅 GH3220 支持）。

1. GH3X2X硬件资源和软件资源要求。

为集成 GH3X2X 驱动库，对 Host 端的硬件资源需求如下：

一个输出 IO，用于硬件复位 GH3X2X。
一个 SPI 接口或 I2C 接口，用于 GH3X2X 通信。
一个 EXTI 接口，用于监控 GH3X2X 中断。
驱动库+DEMO 下不同功能组合的 ROM&RAM 需求：（不包含基本功能和扩展功能）

心率+佩戴		
Rom	Ram	Stack
17266	967	80
心率+血氧+佩戴		
Rom	Ram	Stack
17460	1007	80
心率+血氧+佩戴+ECG		
Rom	Ram	Stack
20712	1455	80
心率+血氧+佩戴+HRV		
Rom	Ram	Stack
17410	1135	80
心率+血氧+佩戴+HRV+ECG		
Rom	Ram	Stack
20784	1539	80

基本功能模块下 ROM&RAM 需求：

模块	Rom	Ram
lead off 检测以及降采样处理	3126	412
单通道调光模块	1541	480
多通道调光模块	2414	1088

扩展功能模块下 ROM&RAM 需求：

模块	Rom	Ram
透传协议（非压缩版本）	6056	8874
透传协议（压缩版本）	8328	9579
无感透传协议（非压缩版本）	6376	16935
无感透传协议（压缩版本）	7470	17839
G-sensor 异步读取处理功能	1732	62

算法空间消耗和实际硬件以及功能启动方式相关联，需要根据《CardiffA 驱动库资源

统计》表格中的“算法空间消耗”Sheet 进行计算。

1.1 GH3X2X SPI接口

当 GH3X2X 中 IICEN 处于低电平状态时，GH3X2X 可以通过 SPI 接口访问。

GH3X2X SPI 是标准的 SPI 通信协议，其中相位为 0（CPHA=0），时钟极性为低电平（CPOL=0）。

GH3X2X SPI 最高时钟速率支持 12MHz；

GH3X2X SPI 支持的操作包含：

1) 16 bits 寄存器读写；
2) 32 bits FIFO 读；
3) 特殊指令。

指令名称	值
写（CMD_WRITE）	0xF0
读（CMD_READ）	0xF1
复位（CMD_RESET）	0xC2
睡眠（CMD_SLP）	0xC4
唤醒（CMD_RESUME）	0xC3

1.2 GH3X2X I2C接口

当 GH3X2X 中 IICEN 处于高电平状态时，GH3X2X 可以通过 I2C 接口访问。

GH3X2X I2C 支持标准的 I2C 协议，支持 Single/Burst 读写操作，采用 7-bit 地址，其中低 2 bits 由外部引脚配置。

GH3X2X I2C 的通信地址对应关系。

MISO	CS	ADDR_Write	ADDR_Read
0	0	0x28	0x29
0	1	0x2A	0x2B
1	0	0x2C	0x2D
1	1	0x2E	0x2F

GH3X2X I2C 最高时钟速率支持 1MHz；

GH3X2X I2C 支持的操作包含：

1) 16 bits 寄存器读写；
2) 32 bits FIFO 读，FIFO 寄存器地址为 0xFFFF；
3) 特殊指令，特殊指令寄存器地址为 0xDDDD。

指令名称	值
写（CMD_WRITE）	0xF0
读（CMD_READ）	0xF1
复位（CMD_RESET）	0xC2
睡眠（CMD_SLP）	0xC4
唤醒（CMD_RESUME）	0xC3

1.3 GH3X2X 中断接口

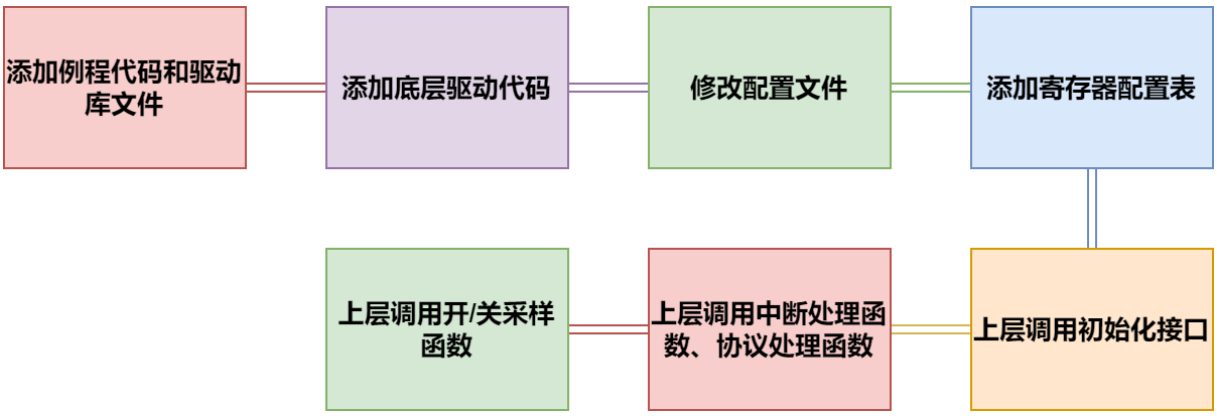
GH3X2X 中断管脚默认状态下为低电平，当存在中断事件需要上报给主控时，GH3X2X 会产生上升沿脉冲，主控检测到上升沿后通知驱动库进行相应的事件处理，超时（20ms）不处理中断事件时会导致持续产生上升沿脉冲，直到事件被处理。

1.4 GH3X2X 复位接口

GH3X2X 复位管脚处于为低电平状态时，芯片进入 DeepSleep 模式，此时所有功能都不可以运行，包括通信。

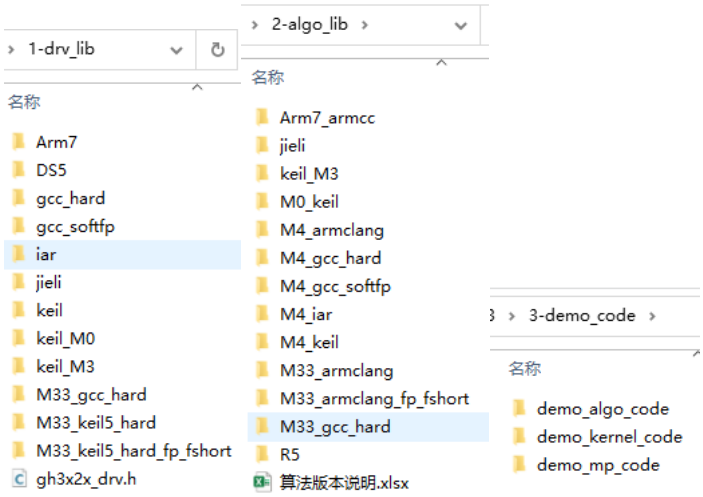
GH3X2X 复位管脚恢复高电平状态时，芯片进入 No Sleep 模式，并产生复位中断，恢复芯片所有配置为默认值。

2. GH3X2X驱动&算法库移植



上图为 GH3X2X 驱动&算法库移植的整体流程，用户需要添加到工程里面的代码主要为驱动库、算法库、demo_algo_code 以及 demo_kernel_code。用户正确添加文件到工程后，将对应平台的外设驱动代码添加到 demo_kernel_code 中，再根据需要的功能修改配置文件，最后实现用户代码里调用顶层接口。

2.1 添加驱动&算法库文件和例程代码到工程



公版的驱动库和算法库包含有多个平台的 lib 库，使用时根据实际平台复制一下对应平台的 lib 库到工程下，demo_code 中包含 demo_algo_code、demo_kernel_code 和 demo_mp_code 三部分代码，其中 demo_mp_code 是本地产测使用的代码，在移植过程

中可以不用移植到工程中。

将 1-drv_lib 中的对应平台驱动库、2-algo_lib 中的对应平台算法库和 3-demo_code 中 demo_algo_code 和 demo_kernel_code 放到工程目录下的 GH3X2X_Code 文件夹中，形成如下的文件夹结构。

```
GH3X2X_Code
 ├──1-drv_lib
 ├──2-algo_lib
 └──3-demo_code
      ├──demo_algo_code
      │    ├──goodix_algo_application
      │    └──goodix_algo_call
      ├──demo_kernel_code
```

2.2 添加底层驱动代码

底层驱动代码主要在 gh3x2x_demo_user.c 文件中，下面按照 Gh3x2xDemoInit 流程逐步实现 GH3X2X 在 nRF52840 上的驱动代码移植。

2.2.1 Log 打印接口

实现基于平台的 LOG 打印函数 GH3X2X_Log。

```
void GH3X2X_Log(GCHAR *log_string)
{
    printk("%s\n", log_string);
}
```

2.2.2 SPI 接口

由于使用 NCS 进行开发，需要修改设备树 SPI 定义以及启用 SPI 功能。

1. 添加修改 nrf52840dk_nrf52840.overlay 文件如下：

```
&pinctrl {
    spi0_default: spi0_default {
        group1 {
            psels = <NRF_PSEL(SPIM_SCK, 1, 13)>,
                <NRF_PSEL(SPIM_MOSI, 1, 14)>,
                <NRF_PSEL(SPIM_MISO, 1, 15)>;
        };
    };
    spi0_sleep: spi0_sleep {
        group1 {
            psels = <NRF_PSEL(SPIM_SCK, 1, 13)>,
                <NRF_PSEL(SPIM_MOSI, 1, 14)>,
                <NRF_PSEL(SPIM_MISO, 1, 15)>;
            low-power-enable;
        };
    };
```

```
};

&spi0 {
    compatible = "nordic,nrf-spi";
    /* Cannot be used together with i2c0. */
    status = "okay";
    clock-frequency = < 4000000 >;
    pinctrl-0 = <&spi0_default>;
    pinctrl-1 = <&spi0_sleep>;
    pinctrl-names = "default", "sleep";
};
```

2. 在 prj.conf 中启用 SPI 模块和 FPU。

```
CONFIG_SPI=y
CONFIG_FPU=y
CONFIG_FP_SOFTABI=y
```

3. 实现基于平台的 SPI 初始化函数、读写函数和 CS 控制函数。

```
#define SPI_DEV      DT_LABEL(DT_NODELABEL(spi0))
#define SPI_CS_DEV   DT_LABEL(DT_NODELABEL(gpio1))
#define SPI_CS_PIN   (12)
#define INT_DEV      DT_LABEL(DT_NODELABEL(gpio1))
#define INT_PIN      (10)
#define RST_DEV      DT_LABEL(DT_NODELABEL(gpio1))
#define RST_PIN      (11)

struct spi_buf_set tx_bufs, rx_bufs;
struct spi_buf txb, rxb;
struct spi_config spi_cfg_single;

struct gh3x2x_dev_info
{
    const struct device *spi_dev;
    const struct device *spi_cs_dev;
    const struct device *int_dev;
    const struct device *rst_dev;
};
static struct gh3x2x_dev_info gh3x2x_info;

void hal_gh3x2x_spi_init(void)
{
    /* init spi and cs pin */
    gh3x2x_info.spi_dev = device_get_binding(SPI_DEV);
    if(NULL == gh3x2x_info.spi_dev)
    {
        EXAMPLE_LOG("Failed to find device %s\n", SPI_DEV);
        return;
```

```
    }
    spi_cfg_single.frequency = 4000000U;
    spi_cfg_single.operation = SPI_OP_MODE_MASTER | SPI_TRANSFER_MSB
                             | SPI_WORD_SET(8) | SPI_LINES_DUAL;
    spi_cfg_single.slave = 0;
    spi_cfg_single.cs = NULL;

    gh3x2x_info.spi_cs_dev = device_get_binding(SPI_CS_DEV);
    if(NULL == gh3x2x_info.spi_cs_dev)
    {
        EXAMPLE_LOG("Failed to find device %s\n", SPI_CS_DEV);
        return;
    }
    gpio_pin_configure(gh3x2x_info.spi_cs_dev, SPI_CS_PIN, GPIO_OUTPUT_HIGH);
    EXAMPLE_LOG("spi init end!\n");
}
```

```
GU8 hal_gh3x2x_spi_write(GU8 write_buffer[], GU16 length)
{
    GU8 ret = 1;
#if (__GH3X2X_SPI_TYPE__ == __GH3X2X_SPI_TYPE_HARDWARE_CS__)
    gpio_pin_set(gh3x2x_info.spi_cs_dev, SPI_CS_PIN,  0);
#endif
    txb.buf = write_buffer;
    txb.len = length;
    tx_bufs.buffers = (const struct spi_buf *)&txb;
    tx_bufs.count = 1U;
    ret = spi_write(gh3x2x_info.spi_dev, &spi_cfg_single, &tx_bufs);
#if (__GH3X2X_SPI_TYPE__ == __GH3X2X_SPI_TYPE_HARDWARE_CS__)
    gpio_pin_set(gh3x2x_info.spi_cs_dev, SPI_CS_PIN,  1);
#endif
    return (ret == 0);
}
```

```
GU8 hal_gh3x2x_spi_read(GU8 read_buffer[], GU16 length)
{
    GU8 ret = 1;
    rxb.buf = read_buffer;
    rxb.len = length;
    rx_bufs.buffers = &rxb;
    rx_bufs.count = 1U;

    ret = spi_read(gh3x2x_info.spi_dev, &spi_cfg_single, &rx_bufs);

    return (ret == 0);
```

```
}
```

```
GU8 hal_gh3x2x_spi_write_F1_and_read(GU8 read_buffer[], GU16 length)
{
    GU8 ret = 1;
    GU8 write_buffer[1] = {0xf1};
    gpio_pin_set(gh3x2x_info.spi_cs_dev, SPI_CS_PIN,  0);

    txb.buf = write_buffer;
    txb.len = 1;
    tx_bufs.buffers = (const struct spi_buf *)&txb;
    tx_bufs.count = 1U;
    ret = spi_write(gh3x2x_info.spi_dev, &spi_cfg_single, &tx_bufs);

    rxb.buf = read_buffer;
    rxb.len = length;
    rx_bufs.buffers = &rxb;
    rx_bufs.count = 1U;

    ret = spi_read(gh3x2x_info.spi_dev, &spi_cfg_single, &rx_bufs);

    gpio_pin_set(gh3x2x_info.spi_cs_dev, SPI_CS_PIN,  1);
    return (ret == 0);
}
```

```
void hal_gh3x2x_spi_cs_ctrl(GU8 cs_pin_level)
{
    GU8 ret = gpio_pin_set(gh3x2x_info.spi_cs_dev, SPI_CS_PIN,  cs_pin_level);
    if (ret < 0) {
        EXAMPLE_LOG("Cannot write gpio");
    }
}
```

2.2.3 I2C 接口

由于使用 NCS 进行开发，需要修改设备树 I2C 定义以及启用 I2C 功能。

1. 添加修改 nrf52840dk_nrf52840.overlay 文件如下：

```
&pinctrl {
    i2c0_default: i2c0_default {
        group1 {
            psels = <NRF_PSEL(TWIM_SDA, 1, 14)>,
                <NRF_PSEL(TWIM_SCL, 1, 13)>;
        };
    };
```

```
    i2c0_sleep: i2c0_sleep {
        group1 {
            psels = <NRF_PSEL(TWIM_SDA, 1, 14)>,
                <NRF_PSEL(TWIM_SCL, 1, 13)>;
            low-power-enable;
        };
    };
};

&i2c0 {
    compatible = "nordic,nrf-twi";
    status = "okay";
    pinctrl-0 = <&i2c0_default>;
    pinctrl-1 = <&i2c0_sleep>;
    pinctrl-names = "default", "sleep";
};
```

2. 在 prj.conf 中启用 I2C 模块和 FPU。

```
CONFIG_I2C=y
CONFIG_FPU=y
CONFIG_FP_SOFTABI=y
```

3. 实现基于平台的 I2C 初始化函数、读写函数。

```
#define I2C_DEV      DT_LABEL(DT_NODELABEL(i2c0))
#define INT_DEV      DT_LABEL(DT_NODELABEL(gpio1))
#define INT_PIN      (10)
#define RST_DEV      DT_LABEL(DT_NODELABEL(gpio1))
#define RST_PIN      (11)

struct spi_buf_set tx_bufs, rx_bufs;
struct spi_buf txb, rxb;
struct spi_config spi_cfg_single;

struct gh3x2x_dev_info
{
    const struct device *i2c_dev;
    const struct device *int_dev;
    const struct device *rst_dev;
};
static struct gh3x2x_dev_info gh3x2x_info;
void hal_gh3x2x_i2c_init(void)
{
    gh3x2x_info->i2c_dev = device_get_binding(I2C_DEV);
    /* code implement by user */
    if (NULL == gh3x2x_info->i2c_dev)
    {
        EXAMPLE_LOG("I2C: Device is not ready.\n");
```

```
        return;
    }
}
```

2.2.4 延时接口

实现基于平台的 us 延时函数和 ms 延时函数，延时时间不要求严格的延时，但是不能小于设定值，us 延时会注册到驱动库中进行使用。

```
void Gh3x2x_BspDelayUs(GU16 usUsec)
{
    k_busy_wait(usUsec);
}
```

```
void Gh3x2x_BspDelayMs(GU16 usMsec)
{
    k_busy_wait(usMsec * 1000);
}
```

2.2.5 复位接口

基于主控平台实现复位管脚的初始化函数和控制函数，函数会注册到驱动库中，由驱动库管理芯片的复位时序。

```
void hal_gh3x2x_reset_pin_init(void)
{
    gh3x2x_info.rst_dev = device_get_binding(RST_DEV);
    if(NULL == gh3x2x_info.rst_dev)
    {
        EXAMPLE_LOG("Failed to find device %s\n", RST_DEV);
        return;
    }
    gpio_pin_configure(gh3x2x_info.rst_dev, RST_PIN, GPIO_OUTPUT_HIGH);
}
```

```
void hal_gh3x2x_reset_pin_ctrl(GU8 pin_level)
{
    GU8 ret = gpio_pin_set(gh3x2x_info.rst_dev, RST_PIN,  pin_level);
    if (ret < 0) {
        EXAMPLE_LOG("Cannot write gpio");
    }
}
```

2.2.6 中断接口

基于主控平台实现中断管脚的初始化，并实现中断回调函数，在回调函数中调用 hal_gh3x2x_int_handler_call_back 函数。

```
void hal_gh3x2x_int_init(void)
{
    gh3x2x_info.int_dev = device_get_binding(INT_DEV);
    if(NULL == gh3x2x_info.int_dev)
    {
        EXAMPLE_LOG("Failed to find device %s\n", INT_DEV);
        return;
    }
    gpio_pin_configure(gh3x2x_info.int_dev, INT_PIN, GPIO_INPUT);
    gpio_init_callback(&gh3x2x_int_callback, gh3x2x_exit_callback, BIT(INT_PIN));
    gpio_add_callback(gh3x2x_info.int_dev, &gh3x2x_int_callback);
    gpio_pin_interrupt_configure(gh3x2x_info.int_dev, INT_PIN, GPIO_INT_EDGE_RISING);
}
```

```
static struct gpio_callback gh3x2x_int_callback;
static void gh3x2x_exit_callback(const struct device *port,
                                 struct gpio_callback *cb, uint32_t pin_msk)
{
    extern void hal_gh3x2x_int_handler_call_back(void);
    hal_gh3x2x_int_handler_call_back();
}
```

2.2.7 与上位机通信接口

Goodix 的 GHealth APP 已经实现通过 BLE 通讯接口进行配置下发、数据采集以及启动功能等操作，可以在前期研发阶段快速测试配置以及后期算法性能有问题时快速采集数据进行分析和优化，由__SUPPORT_PROTOCOL_ANALYZE__宏可以快速开关协议的接入和编译。

同时，为了主控平台和 GHealth 能够正常交互，对服务的 UUID 有明确设定：

1) 服务 UUID： 0000190e-0000-1000-8000-00805f9b34fb
2) 发送属性 UUID： 00000003-0000-1000-8000-00805f9b34fb
3) 接收属性 UUID： 00000004-0000-1000-8000-00805f9b34fb

并且，BLE 通信 MTU 需要修改为 247。

服务快速实现可参考 nus 服务：

1. 将 nus 服务的 nus.c/.h，复制到工程目录下并重命名为 ghealth.c/.h，并将文件中 nus 关键词全部替换为 ghealth，NUS 关键词全部替换为 GHEALTH。
2. 将 LOG_MODULE_REGISTER(bt_nus, CONFIG_BT_NUS_LOG_LEVEL);修改为 LOG_MODULE_REGISTER(bt_ghealth);
3. 修改 ghealth.h 上的 UUID 值如下：

```
/** @brief UUID of the GHEALTH Service. **/
#define BT_UUID_GHEALTH_VAL \
    BT_UUID_128_ENCODE(0x0000190e, 0x0000, 0x1000, 0x8000, 0x00805f9b34fb)

/** @brief UUID of the TX Characteristic. **/
#define BT_UUID_GHEALTH_TX_VAL \
```

```
    BT_UUID_128_ENCODE(0x00000003, 0x0000, 0x1000, 0x8000, 0x00805f9b34fb)

/** @brief UUID of the RX Characteristic. **/
#define BT_UUID_GHEALTH_RX_VAL \
    BT_UUID_128_ENCODE(0x00000004, 0x0000, 0x1000, 0x8000, 0x00805f9b34fb)
```

在 mian 函数内添加服务初始化，在启动广播之前调用 init_ghealth 函数即可完成服务初始化。

```
void bt_ghealth_on_received(struct bt_conn *conn,
            const uint8_t *const data, uint16_t len)
{
    Gh3x2xDemoProtocolProcess(data, len);
}
static int init_ghealth(void)
{
    static struct bt_ghealth_cb ghealth_cb = {
        .received = bt_ghealth_on_received
    };

    int err = bt_nus_init(&ghealth_cb);
    if (err) {
        LOG_ERR("Cannot initialize NUS (err %d)", err);
    }
    return err;
}
```

实现 BLE 通信协议上传接口、定时器初始化接口、定时启动和停止接口。

```
void Gh3x2x_HalSerialSendData(GU8* uchTxDataBuf, GU16 usBufLen)
{
    bt_ghealth_send(NULL, uchTxDataBuf, usBufLen);
}
```

```
void gh3x2x_serial_timer_call(struct k_timer *dummy)
{
    Gh3x2xSerialSendTimerHandle();
}
```

```
void Gh3x2xSerialSendTimerInit(GU8 uchPeriodMs)
{
    k_timer_init(&gh3x2x_serial_timer, gh3x2x_serial_timer_call, NULL);
}
```

```
void Gh3x2xSerialSendTimerStop(void)
{
    k_timer_stop(&gh3x2x_serial_timer);
}
```

```
void Gh3x2xSerialSendTimerStart(void)
{
    k_timer_start(&gh3x2x_serial_timer,
                  K_MSEC(__GH3X2X_PROTOCOL_SEND_TIMER_PERIOD__),
                  K_MSEC(__GH3X2X_PROTOCOL_SEND_TIMER_PERIOD__));
}
```

2.2.8 软件佩戴检测定时器接口

基于主控平台实现软件佩戴定时器的初始化接口、启动和停止接口，为移动检测提供定时器支持。

```
void gh3x2x_adt_timer_call(struct k_timer *dummy)
{
    Gh3x2xDemoMoveDetectTimerHandler();
}
```

```
void Gh3x2xCreateAdtConfirmTimer(void)
{
    k_timer_init(&gh3x2x_adt_timer, gh3x2x_adt_timer_call, NULL);
}
```

```
void Gh3x2x_StartAdtConfirmTimer(void)
{
    k_timer_start(&gh3x2x_adt_timer, K_MSEC(50), K_MSEC(50));
}
```

```
void Gh3x2x_StopAdtConfirmTimer(void)
{
    k_timer_stop(&gh3x2x_adt_timer);
}
```

2.2.9 G-sensor 数据获取接口（示例）

```
void hal_gsensor_start_cache_data(void)
{
    gsensor_clear_buffer();
    gsensor_start_sampling(25);
}
```

```
void hal_gsensor_stop_cache_data(void)
{
    gsensor_clear_buffer();
    gsensor_stop_sampling(25);
}
```

```
void hal_gsensor_drv_get_fifo_data(STGsensorRawdata gsensor_buffer[],
                                                    GU16 *gsensor_buffer_index)
{
    gsensor_read_rawdata_now();

    for (GU16 i = 0; i < gsensor_buffer_cnt(); i++)
    {
        gsensor_buffer[i].sXAxisVal = gsensor512_buffer[i*3 + 0];
        gsensor_buffer[i].sYAxisVal = gsensor512_buffer[i*3 + 1];
        gsensor_buffer[i].sZAxisVal = gsensor512_buffer[i*3 + 2];
    }

    if((*gsensor_buffer_index) > (__GSENSOR_DATA_BUFFER_SIZE__))
    {
        while(1);   // Fatal error !!!
    }
}
```

2.3修改配置文件

1. 打开 gh3x2x_demo_config.h 文件，修改以下内容。

根据实际使用的通信接口配置__GH3X2X_INTERFACE__。

使用 SPI 接口需要修改为：

```
#define __GH3X2X_INTERFACE__                (__GH3X2X_INTERFACE_SPI__)
```

使用 I2C 接口需要修改为：

```
#define __GH3X2X_INTERFACE__                (__GH3X2X_INTERFACE_I2C__)
```

其他需要修改的配置如下：

配置名称	原始值	修改值	备注
__USER_DYNAMIC_DRV_BUF_EN__	1	0	使用静态内存
__FUNC_TYPE_HSM_ENABLE__	1	0	暂不支持该功能
__FUNC_TYPE_BT_ENABLE__	1	0	暂不支持该功能
__FUNC_TYPE_RESP_ENABLE__	1	0	暂不支持该功能
__FUNC_TYPE_AF_ENABLE__	1	0	暂不支持该功能
__FUNC_TYPE_TEST_ENABLE__	1	0	暂不支持该功能
__FUNC_TYPE_PWTT_ENABLE__	1	0	暂不支持该功能
__FUNC_TYPE_BP_ENABLE__	1	0	暂不支持该功能
__FUNC_TYPE_ECG_ENABLE__	1	0	GH302X 芯片不支持 ECG

2. 打开 gh3x2x_demo_algo_config.h 文件，修改以下内容：

配置名称	原始值	修改值	备注
__ALGO_RUN_SIMULTANEOUSLY_SUPPORT__	1	2	算法按功能组串行
__USE_GOODIX_HSM_ALGORITHM__	1	0	暂不支持该算法

__USE_GOODIX_BT_ALGORITHM__	1	0	暂不支持该算法
__USE_GOODIX_RESP_ALGORITHM__	1	0	暂不支持该算法
__USE_GOODIX_AF_ALGORITHM__	1	0	暂不支持该算法
__USE_GOODIX_BP_ALGORITHM__	1	0	暂不支持该算法
__USE_GOODIX_ECG_ALGORITHM__	1	0	GH302X 芯片不支持 ECG

可以参照如下配置组合功能组。

```
#define GH3X2X_ALGO_FUNCTION_GROUP_0 (GH3X2X_FUNCTION_HR|GH3X2X_FUNCTION_SOFT_ADT_GREEN)
#define GH3X2X_ALGO_FUNCTION_GROUP_1 (GH3X2X_FUNCTION_SPO2|GH3X2X_FUNCTION_SOFT_ADT_GREEN)
#define GH3X2X_ALGO_FUNCTION_GROUP_2 (GH3X2X_FUNCTION_HRV|GH3X2X_FUNCTION_SOFT_ADT_GREEN)
#define GH3X2X_ALGO_FUNCTION_GROUP_3 (GH3X2X_FUNCTION_ECG)
#define GH3X2X_ALGO_FUNCTION_GROUP_4 (GH3X2X_NO_FUNCTION)
```

2.4添加寄存器配置表

1. 打开 gh3x2x_demo_reg_array.c 文件，先将文件内部所有的 gh3x2x_reg_list 数组清空，避免后续调试时功能混淆，如下：

```
const STGh3x2xReg gh3x2x_reg_listx[] =
{
    {0,0},
};
```

2. 提供 GH3X2X 模块原理图到代理商处，提出算法功能需求，从而获取寄存器配置表，并将配置表数组按照依次填入 gh3x2x_reg_list 数组中。

2.5应用层调用上层接口

1. 在 main 函数启动 GH3X2X 线程，在 GH3X2X 线程中初始化 GH3X2X 模块，并启用功能，实现中断处理函数。

```
void gh3x2x_task_entry(void){
    Gh3x2xDemoInit();
    Gh3x2xDemoStartSampling(GH3X2X_FUNCTION_ADT);
    while (1){
        k_sleep(K_MSEC(1));
        if(1 == g_uchGh3x2xIntCallBackIsCalled){
            Gh3x2xDemoInterruptProcess();
        }
    }
}
```

2. 按照基本功能要求进行应用逻辑开发，如下示例（HR+NADT+ADT）。

➢ WearEventHook 是硬件佩戴事件的通知函数，可以在 WearEventHook 函数中启动活体功能。

```
void Gh3x2x_WearEventHook(GU16 usGotEvent, GU8 uchExentEx)
{
    if (usGotEvent & GH3X2X_IRQ_MSK_WEAR_OFF_BIT)
    {
        Gh3x2xDemoStopSampling(g_unDemoFuncMode & (~GH3X2X_FUNCTION_ADT));
        EXAMPLE_LOG("Wear off, no object!!!\r\n");
    }
    else if (usGotEvent & GH3X2X_IRQ_MSK_WEAR_ON_BIT)
```

```
    {
#if ((__DRIVER_LIB_MODE__ == __DRV_LIB_WITH_ALGO__) && (__FUNC_TYPE_SOFT_ADT_ENABLE__))
        Gh3x2xDemoStartSampling(GH3X2X_FUNCTION_SOFT_ADT_GREEN | GH3X2X_FUNCTION_HR);
#else
        Gh3x2xDemoStartSampling(GH3X2X_FUNCTION_HR);
#endif
        EXAMPLE_LOG("Wear on, object !!!\r\n");
    }
}
```

- 开启了活体检测功能后，等待活体检测的算法结果，在活体检测的算法结果出来之前，丢弃 HR 的算法结果，活体结果出来后将 HR 算法结果通过 HRS 服务上传。

  NCS 工程创建时使用 zephyr/samples/bluetooth/peripheral_hr 模板创建会自带 HRS 服务，可用于心率数据上传显示。

```
void GH3X2X_SoftAdtGreenAlgorithmResultReport(STGh3x2xAlgoResult * pstAlgoResult,
                                                                GU32 lubFrameId)
{
#if (__USE_GOODIX_SOFT_ADT_ALGORITHM__)
    GH3X2X_SAMPLE_ALGO_LOG_PARAM("[%s]:result = %d,%d\r\n", __FUNCTION__,
                                 pstAlgoResult->snResult[0], pstAlgoResult->snResult[1]);
    //live object
    if (pstAlgoResult->snResult[0] == 0x1)
    {
        extern GU32 g_unDemoFuncMode;
        if((g_unDemoFuncMode & GH3X2X_FUNCTION_HR) != GH3X2X_FUNCTION_HR)
        {
            Gh3x2xDemoStartSampling(GH3X2X_FUNCTION_HR);
        }
         Gh3x2xDemoStopSampling(GH3X2X_FUNCTION_SOFT_ADT_GREEN);
    }
    //non live object
    else if (pstAlgoResult->snResult[0] & 0x2)
    {
        #if __GH_MSG_WTIH_DRV_LAYER_EN__
          GH_SEND_MSG_WEAR_EVENT(GH3X2X_SOFT_EVENT_WEAR_OFF);
        #else
            GH3X2X_SetSoftEvent(GH3X2X_SOFT_EVENT_WEAR_OFF);
        #endif
    }
#endif
}
```

```
void GH3X2X_HrAlgorithmResultReport(STGh3x2xAlgoResult * pstAlgoResult, GU32 lubFrameId)
{
#if (__USE_GOODIX_HR_ALGORITHM__)
```

```
    GH3X2X_SAMPLE_ALGO_LOG_PARAM("[%s]:%dbpm %d %d\r\n",
                        __FUNCTION__, pstAlgoResult->snResult[0],
                                      pstAlgoResult->snResult[1],
                                      pstAlgoResult->snResult[2]);

    extern GU32 g_unDemoFuncMode;
    if((g_unDemoFuncMode & GH3X2X_FUNCTION_SOFT_ADT_GREEN) !=
                                GH3X2X_FUNCTION_SOFT_ADT_GREEN)
    {
        bt_hrs_notify(pstAlgoResult->snResult[0]);
    }

#endif
}
```

2.6测试验证

2.6.1 正常初始化的 LOG 信息输出

1. 最先输出的是驱动库以及扩展功能的软件版本信息。

```
Gh3x2xDemoInit : Democode Version : GH(M)3X2X_DEMO_v1.5_AC_v0.4(build:Oct 28 2022_17:35:30)
Gh3x2xDemoInit : DrvLib Version : v4.0.1.4 (build:Sep 20 2022_18:39:57)
Gh3x2xDemoInit : Protocol Version : Gh3x2x_Uprotocol_v0.4
Gh3x2xDemoInit : Config Version : Gh3x2x_Virtual_Reg_v3.4
```

2. 接下来会输出已使能的算法版本信息，其中未失能的算法版本信息为 no_ver。

```
[gh3x2x_algo]: version = GX_HBA_2.2.0.0(Delta)_S_CONF_nc_goodix_dsp_v1.0.3.2
Gh3x2xDemoInit : Algo Version : GX_HBA_2.2.0.0(Delta)_S_CONF_nc_goodix_dsp_v1.0.3.2
[gh3x2x_algo]: version = HRV_v_2.4.0a
Gh3x2xDemoInit : Algo Version : HRV_v_2.4.0a
[gh3x2x_algo]: version = GX_SPO2_PV_v2.0.1(cardiffA_v2.23_0ae2fc)_nc_goodix_dsp_v1.0.3.2
Gh3x2xDemoInit : Algo Version :
GX_SPO2_PV_v2.0.1(cardiffA_v2.23_0ae2fc)_nc_goodix_dsp_v1.0.3.2
[gh3x2x_algo]: version = Nadt_V1.0.12a
Gh3x2xDemoInit : Algo Version : Nadt_V1.0.12a
[gh3x2x_algo]: version = no_ver
Gh3x2xDemoInit : Algo Version : no_ver
```

3. 再接下来会输出默认配置的 slot 部分信息以及配置版本。

```
[GH3x2xSlotTimeInfo]:CfgIndex[0]:Slot time = 290
[GH3x2xSlotTimeInfo]:CfgIndex[1]:Slot time = 762
[GH3x2xSlotTimeInfo]:CfgIndex[2]:Slot time = 762
[GH3x2xSlotTimeInfo]:CfgIndex[3]:Slot time = 152
[GH3x2xSlotTimeInfo]:CfgIndex[4]:Slot time = 26
[GH3x2xSlotTimeInfo]:CfgIndex[5]:Slot time = 152
Gh3x2xGetConfigVersion : Config Version : 2aeed80f
```

4. 最后输出初始化成功的信息。

```
Gh3x2xDemoInit:init success
```

2.6.2 通信异常的初始化 LOG 信息

输出驱动库以及扩展功能的软件版本信息、算法版本信息后，会输出初始化失败信息，以及失败码。

```
Gh3x2xDemoInit:init fail, error code: -4
```

失败码为 4，是指通信错误，导致错误的常见情况如下：

1. SPI/I2C 的管脚连接错误，例如：SPI 的主机 MSIO 连到了从机的 MOSI。
2. 通信电平不匹配，主控的 IO 输出电平和 GH3X2X 的 VDDIO 电平不一致导致。
3. SPI/I2C 的 API 实现有异常，例如：
   a) 使用到了中断的 API 读写函数，但是在实现 GH3X2X 的读写函数的时候没有等待读写完成标志就退出了函数。
   b) 使用 SPI 的软件 CS 功能配置时，还在读写函数中操作 CS 管脚，导致读写时序异常，GH3X2X 无法正常响应。
4. 没有初始化 GH3X2X 的 RESET 管脚，并且连接到了主控，主控的管脚默认状态为低电平时，会导致 RESET 管脚持续拉低，从而导致 GH3X2X 芯片处于 DeepSleep 状态，无法进行通信。
5. GH3X2X 模块没有正常供电。

2.6.3 测试采样功能是否正常

在保证初始化成功后，并且配置和硬件相对应后，直接启动 HR 功能，观察 LED 能否正常亮绿灯。

出现无法正常亮灯的情况，需要进行以下排查：

1. 检查 LED 是否反接了。
2. 检查 VDDTX 的供电是否正常。
3. VLED 为 3.3V 时检查 VLED 电压是否会有规律性拉低的现象，或者将 VLED 更换到 5V 进行验证是否能够亮灯；部分 LED 的压差较高，GH3X2X 恒流供电时可能会拉低供电导致 LED 不满足压差要求导致无法亮灯。
4. 检查 LDO 的供电电流是否满足要求（>200mA）。

LED 能够正常亮起的时候，打开 gh3x2x_get_rawdata_hook_func 函数中的宏控制，输出采样的原始数据，并将 GH3X2X 模块对空，观察原始数据的值；再用手遮挡 GH3X2X 模块的光窗，观察原始数据的值：

1. 原始数据的值有增大现象，说明 PD 数据基本正常；
2. 原始数据不增大，反而减少，说明对应的 PD 极大可能是焊反了；
3. 原始数据基本没有变化，说明 PD 可能是坏的或者存在虚焊情况导致。

附 录

1. CMake配置修改参考。

打开 CMakeLists.txt 文件，修改为以下内容。

```
cmake_minimum_required(VERSION 3.20.0)

find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})
project(peripheral_hr_gh3x2x_demo)

FILE(GLOB app_sources
```

```
  src/*.c
  src/service/*.c
  src/gh3x2x_code/*.c
  src/gh3x2x_code/demo_kernel_code/src/*.c
  src/gh3x2x_code/demo_kernel_code/src/*.c
  src/gh3x2x_code/demo_algo_code/goodix_algo_call/src/*.c
  src/gh3x2x_code/demo_algo_code/goodix_algo_application/src/*.c
  )

target_sources(app PRIVATE ${app_sources})

include_directories(
  .
  src/service
  src/gh3x2x_code/drv_lib
  src/gh3x2x_code/demo_kernel_code/inc
  src/gh3x2x_code/demo_algo_code/goodix_algo_call/inc
  src/gh3x2x_code/demo_algo_code/goodix_algo_application/inc
  )

zephyr_library_include_directories(${ZEPHYR_BASE}/samples/bluetooth)

target_link_libraries(app PRIVATE
  -L../src/gh3x2x_code/drv_lib/gcc_softfp/_build/gcc_softfp
  -L../src/gh3x2x_code/algo_lib/M4_gcc_softfp

  -Wl,--start-group

  libgh3x2x_drv_cortexM4l_band_gcc_softfp_agc.a
  libgh3x2x_drv_cortexM4l_band_gcc_softfp_ecg.a
  libgh3x2x_drv_cortexM4l_band_gcc_softfp_kernel.a
  libgh3x2x_drv_cortexM4l_band_gcc_softfp_zip.a

  libcortexM4_eabi-gcc_softfp.a
  libcortexM4_eabi-gcc_softfp_hrv.a
  libcortexM4_eabi-gcc_softfp_nadt.a
  libcortexM4_eabi-gcc_softfp_resp.a

  libm.a
  libc.a
  libnosys.a
  libgcc.a

  -Wl,--end-group
)
```